

Массивы. Алгоритмические приемы



Актуализация

- Какие типы данных существует в Java?
- Чем структура данных отличается от управляющей конструкции?
- Что такое стэк?
- Как записать цикл «выведи все четные числа из массива arr» на Python? На Pascal? На 1C? Java?

Типы и структуры данных, управляющие конструкции. Различия

Типы

Какие?

Структуры

Как
организованы

Упр. конст.

Какой
порядок?

1. Для каждого значения укажите подходящий тип в Java:

Значение	Тип в Java
42	
3.14	
true	
"Привет"	
'A'	

1. Для каждого значения укажите подходящий тип в Java (ответ):

Значение	Тип в Java
42	int
3.14	double
true	boolean
"Привет"	String
'A'	char

2. Перевод с Python на Java

Python:

```
a = 10
```

```
b = 3.14
```

```
c = True
```

```
print(a + b)
```

2. Перевод с Python на Java (ответ)

Python:

`a = 10`

`b = 3.14`

`c = True`

`print(a + b)`

Java:

`int a = 10;`

`double b = 3.14;`

`boolean c = true;`

`System.out.println(a + b);`

3. Перевод с Pascal на Python

```
pascal  
var x: integer;  
begin  
  x := 5;  
  writeln(x * 2);  
end.
```

3. Перевод с Pascal на Python (ответ)

pascal

```
var x: integer;  
begin  
  x := 5;  
  writeln(x * 2);  
end.
```

python

```
x = 5  
print(x * 2)
```

4. Какой тип в Java подходит для дробного числа?

- a) int
- b) double
- c) boolean
- d) String

5. В Python тип для True/False — это:

- a) int
- b) bool
- c) boolean
- d) logical

6. В 1С нет разделения на целые и дробные.
Как называется общий тип?

- a) Число
- b) int
- c) float
- d) double

7. Какой тип в Java используется для строки?

- a) string
- b) String
- c) str
- d) char[]

8. Что означает статическая типизация?

- а) тип определяется при запуске
- б) тип известен при объявлении
- с) тип не важен
- д) тип меняется автоматически

9. Сопоставьте

Структура	Принцип
Стек	1. Ключ → значение
Очередь	2. LIFO
Дек	3. FIFO
Словарь	4. Два конца

9. Сопоставьте (ответ)

Структура	Принцип
Стек	2. LIFO
Очередь	3. FIFO
Дек	4. Два конца
Словарь	1. Ключ → значение

10. Перевод с Pascal на Java

```
pascal  
var  
  arr: array[1..3] of integer;  
begin  
  arr[1] := 10;  
  arr[2] := 20;  
  arr[3] := 30;  
  writeln(arr[1] + arr[2]);  
end.
```

Обновление массива на месте

```
char[] chars = {'a', 'b', 'c'};

char temp = chars[0];
chars[0] = chars[chars.length-1];
chars[chars.length-1] = temp;

System.out.println(Arrays.toString(chars));
```

Вывод:

[c, b, a]

Process finished with exit code 0

Пример. Сортировка

```
int[] arr = {-1, 3, 4, 2, 9, 5, 4};  
boolean isSorted = false;  
  
while (!isSorted) {  
    isSorted = true;  
    for (int i = 0; i < arr.length - 1; i++) {  
        if (arr[i] > arr[i + 1]) {  
            int temp = arr[i];  
            arr[i] = arr[i + 1];  
            arr[i + 1] = temp;  
            isSorted = false;  
        }  
    }  
}  
  
System.out.println(Arrays.toString(arr));
```

Вывод:

[-1, 2, 3, 4, 4, 5, 9]

Process finished with exit code 0

Пример. Задача 5: Переместить нули в конец

- Держим указатель `insertPos` — позицию, куда должен встать следующий ненулевой элемент. В цикле идём по массиву указателем `i`. Как только видим ненулевой элемент — меняем его местами с `nums[insertPos]` и сдвигаем `insertPos` вправо.
- Всё, что оказалось слева от `insertPos`, — уже ненулевые в правильном порядке. Всё, что справа, постепенно заполняется нулями.

Два указателя – левый и правый

- Удобно использовать в задаче на проверку палиндрома

```
//           l ->           <- r
int[] arr = {1, 2, 3, 4, 3, 2, 1};

int l = 0;
int r = arr.length - 1;

while (l <= r) {
    // Трассировка
    System.out.println("left: " + arr[l] + ", right: " + arr[r]);
    l++;
    r--;
}
```

Вывод:

```
left: 1, right: 1
left: 2, right: 2
left: 3, right: 3
left: 4, right: 4
```

Process finished with exit code 0

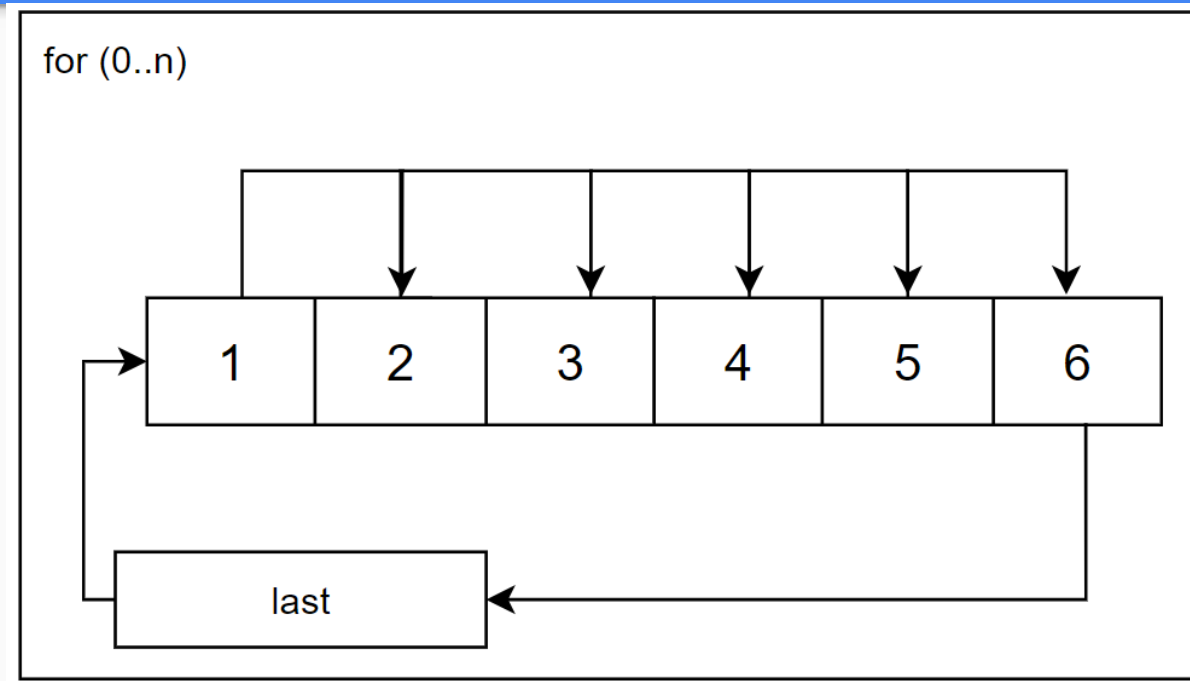
Циклический сдвиг

- $61 \text{ минут} \% 60 \text{ минут в час} = 1 \text{ час } 1 \text{ минута}$



Циклический сдвиг

- n – сдвиг
- last – взять последний элемент
- Нужно идти с конца



Шифр Цезаря и таблица ASCII

- Можно использовать факт того, что буквы закодированы в таблице ASCII (Unicode) и имеют свой код, причем буквы идут в алфавитном порядке.

```
char[] text = "abc".toCharArray();  
for (char c : text) {  
    System.out.println(c + ":" + (int) c);  
}
```

Вывод:

a:97

b:98

c:99

Шифр Цезаря и таблица ASCII

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	\$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

Шифр Цезаря и таблица ASCII

- Чтобы вывести весь алфавит

```
for (char c = 'a'; c < 'z'; c++) {  
    System.out.println(c + ":" + (int) c);  
}
```

- Вывод в 5 столбцов

```
int i = 1;  
for (char c = 'a'; c < 'z'; c++) {  
    System.out.print('\t' + " " + c + ":" + (int) c + " ");  
    if (i % 5 == 0) {  
        System.out.println();  
    }  
    i++;  
}
```

```
a:97  
b:98  
c:99  
d:100  
e:101  
f:102  
g:103
```

a:97	b:98	c:99	d:100	e:101
f:102	g:103	h:104	i:105	j:106
k:107	l:108	m:109	n:110	o:111
p:112	q:113	r:114	s:115	t:116
u:117	v:118	w:119	x:120	y:121

Алгоритм Шифр Цезаря. Вариант 1

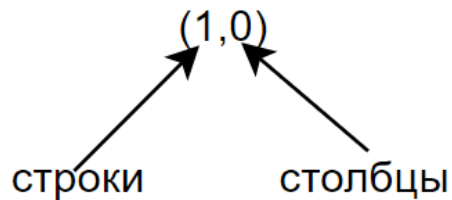
1. Нормализовать сдвиг (т.к. он может быть отрицательным)
2. В цикле взять текущий индекс элемента из таблицы (`text[i]` – 'а')
3. В цикле прибавить к текущему индексу норм. сдвиг
4. Взять остаток на длину алфавита и прибавить 'а' – преобразуем в код из таблицы

Алгоритм Шифр Цезаря. Вариант 2

1. Создать массив с алфавитом
2. Нормализовать сдвиг (т.к. он может быть отрицательным)
3. Создать внешний цикл на количество сдвигов
4. В внутреннем цикле: если встретилась буква алфавита - прибавить к текущему индексу норм. сдвиг и взять остаток на длину алфавита
5. Выйти из внутреннего цикла

Матрицы

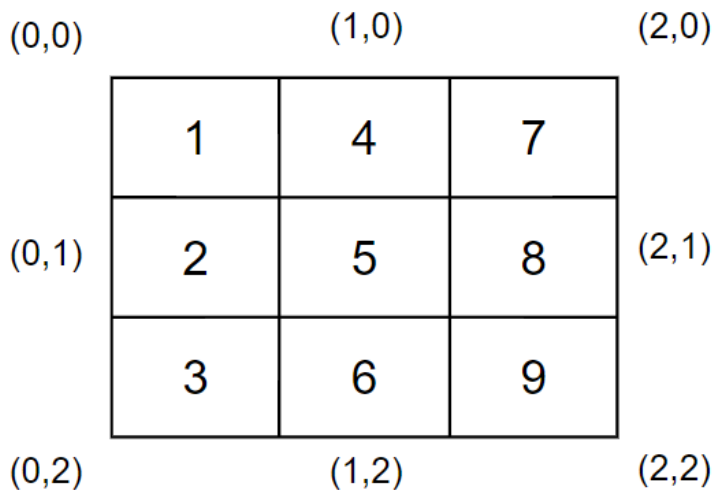
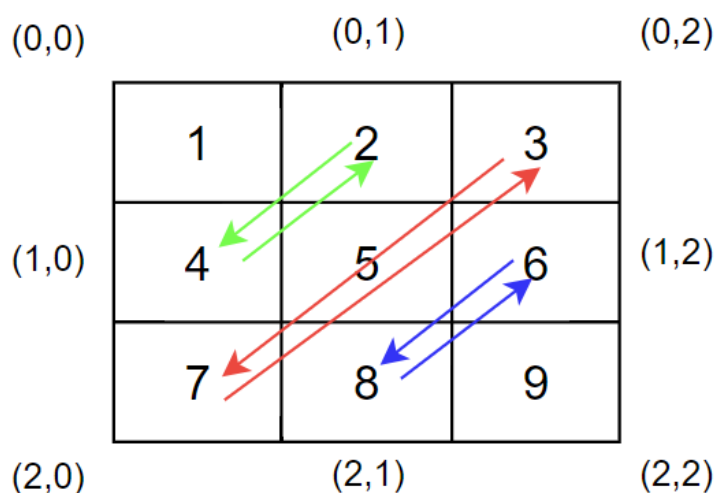
- $(0,1)$ – индексы



(0,0)	(0,1)	(0,2)		
	1	2	3	
(1,0)	4	5	6	(1,2)
	7	8	9	
(2,0)	(2,1)	(2,2)		

Транспонирование

- Столбцы становятся строками, строки столбцами



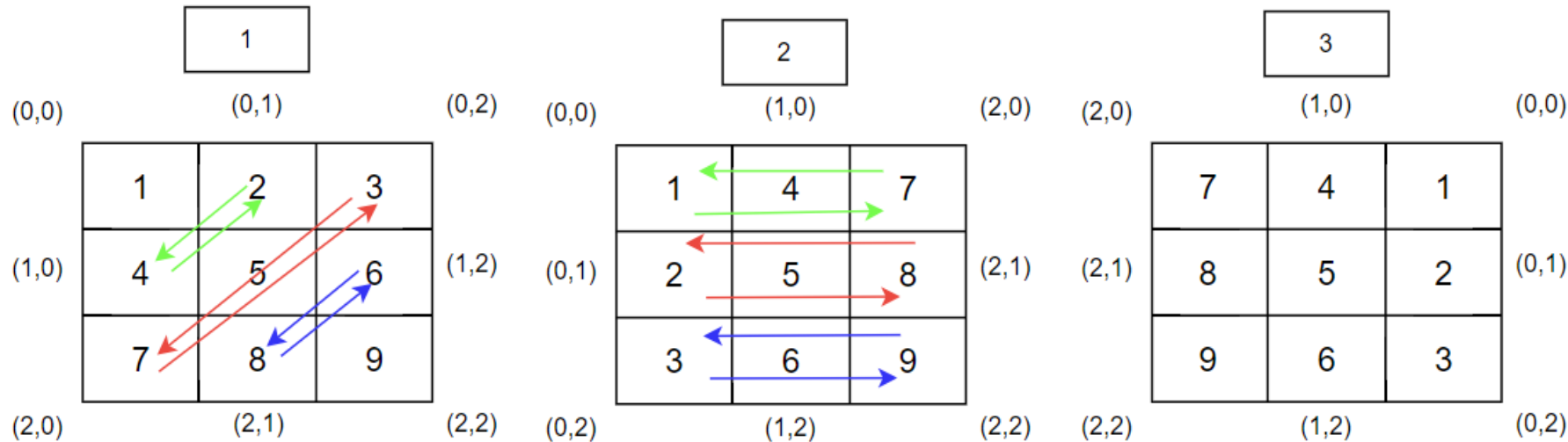
Транспонирование

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};  
  
for (int i = 0; i < matrix.length; i++) {  
    for (int j = i; j < matrix.length; j++) {  
        int temp = matrix[i][j];  
        matrix[i][j] = matrix[j][i];  
        matrix[j][i] = temp;  
    }  
}
```

(i,j)	(0,1)	(0,2)	
	1 •	2 •	3 •
(1,0)	4	5 •	6 •
	7	8	9 •
(2,0)	(2,1)	(2,2)	

Поворот матрицы 90°

1. Транспонирование
2. Разорот каждой строки (2 указателя)



Avg Pooling

(i,j) $n = 1$ $\text{size} = 2*1+1 = 3$ $\text{area} = \text{size}^2 = 9$

(-1,-1)	(-1,0)	(-1,1)	(-1,2)	(-1,3)		
(0,-1)	(0,0)	(0,1)	(0,2)	(0,3)	1	1
(1,-1)	(1,0)	(1,1)	(1,2)	(1,3)	1	1
	1	1	1	1	1	1
	1	1	1	1	1	1
	1	1	1	1	1	1
	1	1	1	1	1	1

(i-n,j-n)	(i-n,j)	(i-n,j+n)				
(i,j-n)	(0,0)	(i,j+n)	1	1	1	1
(i+n,j-n)	(i+n,j)	(i+n,j+n)	1	1	1	1
	1	1	1	1	1	1
	1	1	1	1	1	1
	1	1	1	1	1	1
	1	1	1	1	1	1

Avg Pooling

- 2 внешних цикла по матрице
- 2 внутренних по площади (от $i(j) - n$ до $i(j) + n$) окна с ограничением по высоте и ширине, чтобы не получить выход за границы

RLE (Run-Length Encoding)

- Два указателя: read, write и один счетчик - count
- read — это индекс текущего непрочитанного символа (идёт слева направо)
- write — это индекс следующей свободной позиции в результирующей части массива
- count — считает повторы

Домашнее задание

- Повторить пройденное
- Реализовать утилиту: Блочный шифр